// BACKEND DEVELOPER

조건희

사용자 경험을 고민하며 확장 가능한 서비스를 만드는 것을 좋아합니다.
문제의 원인을 깊이 파고들고 구조적으로 해결하는 백엔드 개발자입니다.

조건희 | devkunhee0817@gmail.com | 010-7917-7975 | https://imkh817.github.io | github.com/imkh817

01. About Me

소개

안녕하세요! 문제의 원인을 깊이 파고들고 구조적으로 해결하는 것을 좋아하는 백엔드 개발자입니다.

현재 개인 블로그 프로젝트를 통해 조회수 처리, 알림 시스템, 검색 성능 개선 등을 구현하며 단순한 기능 구현을 넘어 실제 서비스 환경에서 발생할 수 있는 성능 문제와 설계 문제를 해결하는 경험을 쌓고 있습니다.

최근에는 바이브 코딩에 깊은 관심을 가지고, 주 2회 스터디를 통해 AI 도구를 활용한 개발 방식을 꾸준히 탐구하고 있습니다.

이름	조건희
이메일	devkunhee0817@gmail.com
연락처	010-7917-7975
블로그	https://imkh817.github.io
GitHub	github.com/imkh817

02. Skills

기술 스택

Backend

Java · Spring Boot · Spring Data JPA · QueryDSL · MyBatis · Gradle

Security

Spring Security · JWT Authentication

Database

MySQL · Oracle · Redis

Architecture

REST API · Domain Driven Design · Event Driven Architecture · SSE

Testing

JUnit · Mockito

Infrastructure

AWS S3 · Docker

03. Projects · 01/03 실무 프로젝트

국제약품 LIMS

의약품 품질 관리 공정 전반을 디지털화한 실험실 정보관리 시스템. 시험 의뢰부터 결과 승인, 검체 폐기까지 전 생애주기를 통합 관리하며 식약처 가이드라인을 준수합니다.

주요 성과

- 분산 락 기반 공통 동시성 제어 라이브러리 설계 및 구현
- 다단계 JOIN/UNION ALL 쿼리 튜닝으로 조회 시간 50% 단축 (4.2s → 2.1s)
- 제한적 캐싱 전략 도입으로 참조 데이터 조회 성능 및 시스템 안정성 개선

기술 스택

Java · Spring Boot · Spring Security · Spring Cache · Redis · Oracle · MyBatis · JavaScript

03. Projects · 02/03 실무 프로젝트

하나제약 LIMS

반복적인 시험·검사 업무를 전산화하여 수기 처리 오류를 줄이고, 시험 결과 관리와 승인 프로세스를 일관되게 운영할 수 있도록 지원하는 품질 관리 시스템.

주요 성과

- 재시도·실패 알림 기반 배치 프로그램 안정성 개선
- 도메인 책임 분리 및 전략 패턴 적용으로 복잡한 비즈니스 로직 구조 개선

기술 스택

Java · Spring Boot · Spring Security · Oracle · MyBatis · JavaScript

03. Projects · 03/03 사이드 프로젝트

개인 블로그 플랫폼

직접 사용하기 위해 만든 블로그 서비스. 단순 기능 구현을 넘어 성능 병목, 동시성, 보안 이슈를 직접 발견하고 해결하는 과정에 집중했습니다.

주요 성과

- 게시글 검색 API 최적화 (응답속도 11배 개선)
- Redis 조회수 동시성 및 정합성 처리
- SSE 실시간 알림 (오프라인 유실 방지 포함)
- JWT 보안 단계적 강화 (블랙리스트 + Rotation)

기술 스택

Spring Boot · Java · MySQL · Redis · Spring Security · Spring Data JPA · QueryDSL · AWS S3 · JUnit · Test Container

// DEEP DIVE

프로젝트 상세

각 프로젝트에서 발견하고 해결한
기술적 문제들을 담은 심층 문서입니다.

국제약품 LIMS · 성과 3건 | 하나제약 LIMS · 성과 2건 | 개인 블로그 플랫폼 · 성과 5건

실무 프로젝트 — 상세

국제약품 LIMS

의약품 품질 관리 공정 전반을 디지털화하여 시험 의뢰부터 분석, 결과 승인, 검체 폐기까지의 전 생애주기를 통합 관리하는 실험실 정보관리 시스템.

식약처 가이드라인을 준수하는 감사 추적(Audit Trail)과 ERP 연동을 통해 품질 데이터의 신뢰성과 전사 시스템 간 데이터 정합성을 안정적으로 확보합니다.

Java · Spring Boot · Spring Security · Spring Cache · Redis · Oracle · MyBatis · JavaScript

담당 역할

Backend Developer로 참여하여 LIMS의 핵심 품질 관리 기능과 외부 시스템 연동 영역을 담당했습니다.

시험 데이터 처리와 승인 흐름에서 발생할 수 있는 동시성 및 정합성 이슈를 분석하고, 운영 환경에서도 안정적으로 동작하도록 개선하는 작업에 집중했습니다.

01

국제약품 LIMS

중복 승인·상태 꼬임 문제 해결을 위한 공통 동시성 제어 설계

PROBLEM

운영 환경에서 동일 검체에 대해 여러 사용자가 동시에 승인·폐기·상태 변경 요청을 보내면서 상태가 중복 처리되거나 이전 상태로 덮어써지는 문제가 반복적으로 발생했습니다.

RESULT

중복 승인 및 상태 꼬임 이슈가 완전히 해결되었습니다.

승인·상태 변경 과정에서 발생하던 데이터 불일치가 제거되어, 운영 중 수동 데이터 보정 작업이 더 이상 필요하지 않게 되었습니다.

APPROACH

01. LIMS 프로젝트마다 상태 변경·승인 흐름에서 동시성 위험이 공통으로 존재함을 파악
02. 각 서비스마다 개별 구현 시 유지보수 비용 증가 및 처리 방식의 불일치 우려
03. 동시성 제어 로직을 공통 라이브러리로 분리 — 락 획득 → 실행 → 재시도 → 락 해제 고정 흐름 정의
04. Redis 기반 분산 락으로 다중 인스턴스 환경 지원, 락 키·대기 시간·재시도 전략을 유연하게 설정 가능하도록 설계

Redis · Distributed Lock · Spring Boot · Library Design

GitHub ↗

02 국제약품 LIMS

다단계 JOIN / UNION ALL 기반 보고서 쿼리 튜닝으로 조회 시간 50% 단축

PROBLEM

여러 기준 데이터를 조합해 한 번에 보여주는 보고서 쿼리가 다수의 테이블 JOIN과 UNION ALL로 구성되어 있었습니다.

데이터 누적에 따라 조회 시간이 점점 늘어나 운영 환경에서 병목으로 작용했습니다.

RESULT

동일 조건 기준 평균 조회 시간이 4.2초 → 2.1초로 약 50% 단축되었습니다.

보고서 조회 시 체감 지연이 개선되었습니다.

APPROACH

01. 단순 인덱스 추가만으로는 개선이 미미했고, 실행 계획 분석을 통해 근본 원인을 파악
02. 불필요한 JOIN 제거 및 위치 재조정으로 중간 결과 집합 크기 감소
03. UNION ALL로 합쳐진 쿼리 중 공통 조건을 선필터링하도록 구조 변경
04. WHERE 절에서 반복적으로 사용되는 주요 조건 컬럼에 인덱스 추가로 Full Scan 제거

Oracle · Query Optimization · Execution Plan · Index

03

국제약품 LIMS

제한적 캐싱 도입으로 조회 성능 및 시스템 안정성 개선

PROBLEM

공통 코드와 품목 마스터처럼 변경 빈도가 낮은 참조 데이터가 있음에도 화면 조회나 업무 처리 시마다 데이터베이스를 직접 조회하는 구조였습니다.

시험 의뢰 등록이나 결과 입력 같은 주요 기능에서 동일한 기준 데이터가 반복 조회되어 DB 부하 증가와 응답 지연이 발생했습니다.

RESULT

반복적인 참조 데이터 조회로 인한 불필요한 DB 접근이 감소했습니다.

다수 사용자가 동시에 접근하는 환경에서 화면 응답 속도와 시스템 안정성이 개선되었습니다.

APPROACH

01. LIMS는 시험 결과·승인 상태 등 실시간 정합성이 중요한 데이터가 많아 전체 캐싱은 부적합하다고 판단
02. 변경 가능성이 낮고 조회 빈도가 높은 코드성·마스터 데이터만 캐싱 대상으로 선별
03. 데이터 변경 시 캐시를 직접 갱신하는 대신 Evict 방식으로 제거 후 재조회 시 DB 기준으로 재적재
04. 캐시-DB 간 데이터 불일치 및 동시성 상황에서의 리스크 최소화

Redis · Spring Cache · Cache Evict · Performance

실무 프로젝트 – 상세

하나제약 LIMS

반복적인 시험·검사 업무를 전산화하여 수기 처리로 인한 오류를 줄이고, 시험 결과 관리와 승인 프로세스를 일관되게 운영할 수 있도록 지원하는 품질 관리 시스템.

감사 추적 기반의 이력 관리로 데이터 신뢰성을 확보하며 실제 운영 환경에서 활용되고 있습니다.

Java · Spring Boot · Spring Security · Oracle · MyBatis · JavaScript

담당 역할

Backend Developer로 참여하여 하나제약 LIMS의 품질 관리 핵심 기능을 개발했습니다.

시험 의뢰·결과·승인 등 주요 도메인 로직과 ERP/MES 인터페이스 API를 개발하며, 데이터 정합성과 트랜잭션 안정성을 중점적으로 개선했습니다.

01

하나제약 LIMS

운영 배치 프로그램 안정성 개선

PROBLEM

운영 중이던 배치 프로그램은 장비 상태를 일괄 갱신하고, 다음날 수행될 모니터링 시험을 자동 생성하는 역할을 담당하고 있었습니다.

DB 부하나 네트워크 지연 같은 일시적인 오류가 발생할 경우 배치가 즉시 실패·종료되는 구조였습니다.

실패 여부를 즉시 인지할 수단이 없어 운영 단계에서 로그를 직접 확인해야만 상황을 파악할 수 있었습니다.

RESULT

일시적인 장애에도 배치가 자동으로 복구를 시도하는 구조가 확보되었습니다.

반복 실패 시 즉시 알림을 통해 운영자가 빠르게 대응할 수 있게 되었습니다.

APPROACH

01. 기존 Spring @Scheduled 기반 구조를 유지해 변경 범위를 최소화
02. 예외 발생 시 즉시 종료하지 않고 자동 재시도 수행, 최대 3회로 제한하여 무한 재시도 방지
03. 재시도 간 간격을 두어 일시적인 장애의 자연 회복 가능성 확보
04. 재시도 이후에도 실패 시 배치 식별 정보·실행 시각·실패 원인을 포함한 회사 메일 알림 발송
05. Quartz 등 별도 스케줄러 도입 없이 기존 구조를 유지하여 도입 복잡도 최소화

Spring Batch · @Scheduled · Retry · Mail Notification

02

하나제약 LIMS

도메인 책임 분리 및 전략 패턴 적용을 통한 복잡한 비즈니스 로직 개선

PROBLEM

LIMS 특성상 하나의 기능에 5~10개 이상의 테이블이 연관되고, 시험 유형·판정 방식·상태에 따른 다수의 조건 분기가 존재했습니다.

초기 구조에서는 상태 변경 규칙이 여러 서비스에 분산되어 중복과 불일치 가능성이 있었습니다.

서비스 레이어에 if-else 분기가 누적되어 메서드가 비대해지고 변경 시 영향 범위 예측이 어려웠습니다.

RESULT

서비스 레이어에 집중되던 조건 분기와 상태 변경 로직이 도메인과 전략 클래스로 분산되어 가독성과 변경 용이성이 개선되었습니다.

신규 시험 유형 추가 시 기존 코드 수정 없이 전략 클래스 추가만으로 확장 가능한 구조가 되었습니다.

APPROACH

01. 상태와 규칙을 가진 도메인이 스스로 상태를 변경하도록 구조 재설계
02. 시험 의뢰·시험 결과 등 상태를 가지는 개념에 상태 전이 규칙과 검증 로직을 도메인 내부로 이동
03. 서비스는 상태 변경을 "요청"만 하고, 변경 가능 여부 판단과 전이 규칙은 도메인이 책임지도록 분리
04. 시험 유형별 결과 처리 로직을 공통 인터페이스로 정의하고 유형별 전략 클래스로 분리 (전략 패턴 적용)
05. 조건 분기가 늘어나도 기존 코드 수정 없이 전략 클래스 추가만으로 확장 가능한 구조 확보

Domain Design · Strategy Pattern · OCP · Refactoring

사이드 프로젝트 — 상세

개인 블로그 플랫폼

직접 사용하기 위해 만든 블로그 서비스.

단순 기능 구현을 넘어 성능 병목, 동시성, 보안 이슈를 직접 발견하고 해결하는 과정에 집중했습니다.

테스트 코드로 문제를 재현하고, 지표로 결과를 검증하는 방식으로 개발했습니다.

Spring Boot · Java · MySQL · Redis · Spring Security · Spring Data JPA · QueryDSL · AWS S3 · JUnit · Test Container

주요 기능

- 게시글 작성 / 검색 / 정렬 / 임시저장
- 댓글 및 대댓글, 좋아요, 구독
- SSE 기반 실시간 알림
- JWT 인증 및 RefreshToken 관리
- Redis 기반 조회수·좋아요 카운트 처리
- Post-Comment Aggregate 분리 설계

01

개인 블로그 플랫폼

게시글 검색 API 최적화 — fetch join 메모리 페이징 발견 및 해결

PROBLEM

N+1 문제를 해결하기 위해 fetch join을 적용했고, 기능 테스트는 정상이었습니다.

그러나 성능 테스트 중 Hibernate 경고 로그와 함께 실제 SQL에 LIMIT 절이 적용되지 않는 것을 발견했습니다.

게시글 100,000건·태그 300,000건 기준으로 단 5개의 요청만으로 Heap이 1.38GB까지 치솟고, 평균 응답 시간이 8.2초에 달했습니다.

ROOT CAUSE

1:N 컬렉션 fetch join과 페이징을 함께 사용하면, Hibernate는 데이터 정합성을 위해 LIMIT을 제거하고 전체 Row를 메모리에 올린 뒤 애플리케이션 단에서 페이징을 수행합니다.

데이터가 늘수록 OOM 위험이 선형적으로 증가하는 구조였습니다.

RESULT

평균 응답 시간 8,193ms → 729ms (11.2배 향상)

Heap 사용량 1.38GB → 76MB (94% 감소), CPU 점유율 80% → 20% (75% 절감)

처리량 32.7/min → 4.5/sec (약 8.2배 향상)

APPROACH

01. fetch join 제거 → DB 레벨에서 LIMIT/OFFSET이 동작하도록 변경
02. default_batch_fetch_size: 100 설정으로 지연 로딩 시 IN 절 묶음 조회 적용하여 N+1 해소
03. JMeter + VisualVM으로 개선 전후 Heap·CPU·응답 시간 지표 비교 검증

응답 시간	Heap 사용량	CPU 점유율
~~8,193ms~~	~~1.38GB~~	~~80%~~
729ms	**76MB**	**20%**
11.2배 향상	94% 감소	75% 절감

JPA · QueryDSL · BatchSize · JMeter · VisualVM

02

개인 블로그 플랫폼

Redis 조회수 flush 중 동시성으로 인한 카운트 유실 문제 해결

PROBLEM

조회수를 Redis에 누적 후 5분마다 DB에 flush하는 구조를 구현했습니다.

동시성 테스트 중, flush가 GET으로 값을 읽은 직후 새로운 조회 요청이 INCR을 수행하고, 이후 flush가 DEL을 실행하면서 중간에 들어온 조회수 +1이 유실되는 상황을 발견했습니다.

ROOT CAUSE

GET과 DEL이 별개의 명령어라 두 연산 사이에 다른 요청이 끼어들 수 있었습니다.

원자적 처리가 보장되지 않는 구조였습니다.

RESULT

GET과 DEL 사이에 다른 요청이 끼어드는 race condition이 원천 차단되어 조회수 유실 문제가 해소되었습니다.

APPROACH

01. Redis 공식 문서에서 GETDEL 명령어 확인 — 조회와 삭제를 하나의 원자적 연산으로 처리
02. flush 로직의 GET + DEL을 GETDEL(getAndDelete)로 교체
03. 동일 동시성 시나리오 테스트 재실행으로 유실 없음 검증

Redis · GETDEL · Atomic Operation · Concurrency

03 개인 블로그 플랫폼

SSE 실시간 알림 — 오프라인 유실 방지 및 트랜잭션 정합성 확보

PROBLEM

① 사용자가 오프라인 상태일 때 발생한 알림이 SSE Emitter가 없어 그냥 무시됐습니다.

② Notification을 DB에 저장하는 트랜잭션이 커밋되기 전에 SSE가 먼저 전송되어, 클라이언트가 알림 목록을 조회하면 빈 결과가 반환되는 타이밍 문제가 있었습니다.

ROOT CAUSE

① 알림을 전송만 하고 저장하지 않는 구조라 오프라인 사용자의 알림이 보관되지 않았습니다.

② REQUIRES_NEW 트랜잭션이 커밋되기 전에 SSE가 발송되는 실행 순서 문제였습니다.

RESULT

오프라인 알림 유실 문제가 해소되었습니다.

SSE 수신 후 즉시 알림을 조회했을 때 빈 결과가 반환되는 타이밍 이슈도 함께 해소되었습니다.

APPROACH

01. 구독 이벤트 발생 시 Notification 테이블에 먼저 저장 후 SSE 전송 시도하는 구조로 변경
02. SSE 전송을 AFTER_COMMIT 기반 별도 이벤트로 분리 → Notification DB 커밋 완료 이후에만 SSE 발송
03. REQUIRES_NEW 트랜잭션으로 알림 저장 실패가 구독 데이터에 영향을 주지 않도록 격리

SSE · Spring Event · AFTER_COMMIT · REQUIRES_NEW · TransactionalEventListener

04 개인 블로그 플랫폼

JWT 보안 단계적 강화 — 블랙리스트 + RefreshToken Rotation

PROBLEM

① 로그아웃 후에도 기존 Access Token으로 API 호출이 그대로 성공했습니다.

② RefreshToken 재발급 후 기존 토큰이 무효화되지 않아, 탈취된 RefreshToken으로 무한정 Access Token 재발급이 가능했습니다.

ROOT CAUSE

JWT는 stateless하여 서버가 발급한 토큰을 별도로 추적하지 않습니다.

프론트에서 토큰을 삭제해도 서버 입장에서는 아무런 상태 변화가 없어, 유효 서명과 만료시간만 통과하면 인증이 허용됩니다.

RESULT

로그아웃 후 토큰 재사용이 차단되었습니다.

RefreshToken 탈취 시 한 번 사용 후 이전 토큰이 즉시 무효화되어, 피해 범위가 최소화되는 구조가 확보되었습니다.

APPROACH

01. 로그아웃 시 해당 Access Token을 Redis 블랙리스트에 등록, TTL을 토큰 남은 만료시간으로 설정 → 만료 시 자동 삭제
02. Access Token 만료 시 자동 갱신을 위한 RefreshToken 도입 (Access: 30분, Refresh: 7일)
03. Refresh Token Rotation 적용 — 재발급 시 기존 토큰 즉시 무효화, 이전 토큰 재사용 시 TokenTamperedException 발생
04. 각 단계마다 테스트 코드로 취약점을 직접 재현한 뒤 해결 여부를 검증

JWT · Redis · Blacklist · Refresh Token · Token Rotation

05 개인 블로그 플랫폼

좋아요 정렬 요구사항으로 인한 설계 재검토 — Redis 캐시 → 비정규화

PROBLEM

좋아요 N+1 문제를 해결하기 위해 "관계 데이터는 DB, 카운트는 Redis"로 분리하는 구조를 설계했습니다.

그런데 "좋아요 순 정렬" 요구사항이 추가되면서 문제가 생겼습니다. DB의 ORDER BY는 Redis를 참조할 수 없어 COUNT 서브쿼리가 매 정렬마다 실행되었고, Redis를 도입한 이유 자체가 사라졌습니다.

ROOT CAUSE

DB에도 likeCount 컬럼이 필요하고 Redis에도 likeCount를 유지하면, 두 저장소가 같은 숫자를 각자 최신으로 유지해야 하는 정합성 관리 부담이 발생합니다.

정렬 요구사항이 추가된 시점에서 Redis가 해결해주는 것보다 관리해야 할 것이 더 많아졌다고 판단했습니다.

RESULT

좋아요 순 정렬이 컬럼 직접 참조로 단순해지고, 서브쿼리가 제거되어 정렬 성능이 확보되었습니다.

PostLikeService가 Post 내부 구현에 의존하지 않는 구조로 개선되었습니다.

APPROACH

01. Post 테이블에 like_count 컬럼 추가 (비정규화) — 정렬과 조회를 하나의 컬럼으로 통일
02. PostLikeService가 Post 애그리거트를 직접 수정하는 문제를 인지 → AFTER_COMMIT 이벤트로 카운트 갱신 분리
03. REQUIRES_NEW 트랜잭션으로 PostLike와 Post를 다른 트랜잭션에서 처리하여 애그리거트 경계 유지

Domain Design · AFTER_COMMIT · REQUIRES_NEW · Denormalization · Aggregate